Clase 9: Modularización y Bibliotecas Externas: De un Script a un Proyecto

Introducción: La Arquitectura del Software

Hasta este punto, hemos trabajado exclusivamente con notebooks de Jupyter en Google Colab, utilizando archivos en formato .ipynb. Esto ha sido perfecto para aprender, pero a medida que nuestros programas crecen en complejidad, este enfoque se vuelve insostenible. Imaginen un libro de recetas de cocina con una sola página de 500 metros de largo. Encontrar algo, corregir un ingrediente o reutilizar una preparación base sería una pesadilla.

Hoy vamos a dar un salto fundamental: aprenderemos a organizar nuestro código como lo hacen los profesionales, **dividiéndolo en partes lógicas y reutilizables llamadas módulos**. Dejaremos de escribir *scripts* para empezar a construir *proyectos* con una arquitectura sólida. Además, abriremos la puerta al inmenso ecosistema de Python, aprendiendo a usar las "herramientas" que ya vienen incluidas en su biblioteca estándar.

Objetivos de la Clase:

Al finalizar esta sesión, serás capaz de:

- **Justificar** la necesidad de dividir un programa complejo en múltiples archivos.
- **Crear** tus propios módulos de Python para encapsular y reutilizar funcionalidades.
- **Importar** funciones y variables desde tus propios módulos a un script principal.
- **Comprender y aplicar** la construcción `if __name__ == "__main__:"` para crear código dual (reutilizable y ejecutable).
- **Diagnosticar y resolver** errores comunes de importación como `ModuleNotFoundError`.
- **Explorar y utilizar** módulos clave de la biblioteca estándar de Python (`math`, `random`, `datetime`, `os`).
- **Refactorizar** un programa de un solo archivo a una estructura de proyecto modular profesional.

---

1. El Problema: El "Código Espagueti" en un Solo Archivo

Cuando todo nuestro código reside en un único archivo, nos enfrentamos a varios problemas a medida que crece:

- **Dificultad de lectura:** Cientos o miles de líneas de código sin una estructura clara son difíciles de seguir.
- **Baja Reutilización:** Si tienes una función útil en `programa_A.py` y la necesitas en `programa_B.py`, la única opción es copiar y pegar. ¿Y si encuentras un error? Tienes que acordarte de corregirlo en *todas* las copias.

- **Colaboración Imposible:** Si dos personas quieren trabajar en el mismo archivo a la vez, la coordinación es un caos.
- **Mantenimiento Complejo:** Arreglar un bug se convierte en una cirugía a corazón abierto, con riesgo de romper otras partes sin darte cuenta.

Refuerzo Visual: De Monolito a Módulos

Imaginemos nuestro código. Podemos tenerlo de dos maneras:

Antes: Un solo archivo gigante (El Monolito)

```
# 📄 mi_programa_gigante.py
# --- Funciones de cálculo ---
def calcular_promedio(...): ...
def calcular_maximo(...): ...
# --- Funciones de archivos ---
def leer_csv(...): ...
def guardar_reporte(...): ...
# --- Lógica principal ---
datos = leer_csv(...)
promedio = calcular_promedio(datos)
maximo = calcular_maximo(datos)
guardar_reporte(promedio, maximo)
# ... y así por 500 líneas más
```

Después: Un proyecto organizado (Modular)

```
# 📂 mi_proyecto/
# ├── 📄 main.py (El Orquestador)
# ├── 📄 calculos.py (Módulo de Cálculos)
# └── 📄 gestor_archivos.py (Módulo de I/O)
```

`En main.py:`

La segunda versión es más limpia, cada archivo tiene una **responsabilidad única** y el `main.py` se lee casi como un resumen de lo que hace el programa.

```
# En main.py:
import calculos
import gestor_archivos

datos = gestor_archivos.leer_csv(...)
promedio = calculos.calcular_promedio(datos)
maximo = calculos.calcular_maximo(datos)
gestor_archivos.guardar_reporte(promedio, maximo)
```

`calculos.py`

Este módulo se enfoca exclusivamente en operaciones matemáticas. No sabe de dónde vienen los datos (si de un archivo, de internet, etc.), simplemente los recibe y los procesa.

```
# 📂 mi_proyecto/
# └── 📄 calculos.py

def calcular_promedio(lista_numeros):
    """
    Calcula el promedio de una lista de números.

    Args:
        lista_numeros (list): Una lista de enteros o flotantes.

    Returns:
        float: El promedio de los números. Devuelve 0 si la lista está
vacía.
    """
    if not lista_numeros:
        return 0

    total = sum(lista_numeros)
    return total / len(lista_numeros)

def calcular_maximo(lista_numeros):
    """
    Encuentra el valor máximo en una lista de números.

    Args:
        lista_numeros (list): Una lista de enteros o flotantes.

    Returns:
        float or int: El número más alto de la lista. Devuelve 0 si la
lista está vacía.
    """
    if not lista_numeros:
        return 0

    return max(lista_numeros)

# --- Bloque de prueba ---
```

```
# Este código solo se ejecuta si corremos "python calculos.py"
directamente.
# Es ideal para probar que las funciones del módulo funcionan
correctamente.
if __name__ == "__main__":
    print("Ejecutando pruebas para el módulo de cálculos...")
    datos_prueba = [10, 20, 30, 40, 50]

    promedio_esperado = 30.0
    maximo_esperado = 50

    promedio_calculado = calcular_promedio(datos_prueba)
    maximo_calculado = calcular_maximo(datos_prueba)

    print(f"Datos de prueba: {datos_prueba}")
    print(f"Promedio calculado: {promedio_calculado} (Esperado:
{promedio_esperado})")
    print(f"Máximo calculado: {maximo_calculado} (Esperado:
{maximo_esperado})")

    # Prueba con lista vacía
    print("\nProbando con una lista vacía...")
    print(f"Promedio: {calcular_promedio([])}")
    print(f"Máximo: {calcular_maximo([])}")
```

gestor_archivos.py

Este módulo se encarga de toda la interacción con el sistema de archivos (entrada/salida o I/O). No realiza ningún cálculo, solo lee y escribe datos.

```
# 📂 mi_proyecto/
# └── 📄 gestor_archivos.py

import pandas as pd

def leer_csv_columna(ruta_archivo, nombre_columna):
    """
    Lee un archivo CSV usando pandas y extrae una columna como una
lista.

    Args:
        ruta_archivo (str): La ruta al archivo CSV.
        nombre_columna (str): El nombre exacto de la columna a extraer.

    Returns:
        list: Una lista con los datos de la columna.
```

```
            Devuelve una lista vacía si ocurre un error.
    """
    try:
        # pandas lee el CSV directamente a un objeto llamado DataFrame
        df = pd.read_csv(ruta_archivo)

        # Seleccionamos la columna por su nombre y la convertimos a una
lista
        datos = df[nombre_columna].tolist()
        return datos

    except FileNotFoundError:
        print(f"Error: El archivo '{ruta_archivo}' no fue encontrado.")
        return []
    except KeyError:
        print(f"Error: La columna '{nombre_columna}' no fue encontrada
en el archivo.")
        return []
    except Exception as e:
        print(f"Ocurrió un error inesperado al leer el archivo con
pandas: {e}")
        return []

def guardar_reporte(ruta_archivo, promedio, maximo):
    """
    Guarda los resultados del análisis en un archivo de texto.

    Args:
        ruta_archivo (str): La ruta donde se guardará el reporte.
        promedio (float): El valor del promedio a guardar.
        maximo (float or int): El valor máximo a guardar.
    """
    with open(ruta_archivo, 'w', encoding='utf-8') as archivo:
        archivo.write("--- Reporte de Análisis de Datos ---\n")
        archivo.write("="*35 + "\n\n")
        archivo.write(f"Promedio de los datos: {promedio:.2f}\n")
        archivo.write(f"Valor máximo encontrado: {maximo}\n")
```

---

2. La Solución: Módulos, ¡Nuestros Bloques de LEGO!

En Python, la solución es elegantemente simple: **cualquier archivo `.py` es un módulo**.

Un módulo es un archivo que contiene definiciones de Python (funciones, variables, etc.). La idea es agrupar código con funcionalidades relacionadas. Para usar el contenido de un módulo en otro archivo, usamos la palabra clave **`import`**.

Guía Práctica para Modularizar (Checklist):

1. ✅ **Identificar Responsabilidades:** Mira tu código. ¿Qué partes se encargan de los cálculos? ¿Qué partes manejan archivos? ¿Qué partes interactúan con el usuario?
2. ✅ **Extraer a Funciones:** Asegúrate de que esas partes estén encapsuladas en funciones bien definidas.
3. ✅ **Agrupar en un Módulo:** Mueve las funciones relacionadas a un nuevo archivo `.py` (ej: `utils.py`, `procesamiento.py`). ¡Este es tu nuevo módulo!
4. ✅ **Importar y Usar:** En tu archivo principal, usa `import nombre_del_modulo` y luego llama a las funciones con `nombre_del_modulo.nombre_funcion()`.

---

3. Refactorizando el Ejercicio de la Clase 8 (Aplicando el Checklist)

Tomemos el "Filtro de Productos" de la clase anterior. Leía `inventario.csv` y escribía en `productos_a_reponer.txt` los productos con stock bajo (< 20).

Paso 1 y 2: Identificar y Extraer

- **Responsabilidad 1:** Leer y filtrar datos del CSV. Esto puede ser una función `filtrar_productos(...)`.
- **Responsabilidad 2:** Escribir el reporte en un archivo de texto. Esto puede ser `generar_reporte(...)`.

Paso 3: Agrupar en un Módulo Creamos un archivo `procesamiento.py` y movemos estas funciones allí.

`procesamiento.py` (nuestro módulo de lógica):

```
import csv

def filtrar_productos_por_stock(ruta_archivo_csv, stock_minimo):
    """
    Lee un archivo CSV de inventario y devuelve una lista de nombres de
```

```
productos
    cuyo stock está por debajo del umbral especificado.
    """
    productos_filtrados = []
    with open(ruta_archivo_csv, 'r') as archivo_csv:
        lector_csv = csv.reader(archivo_csv)
        next(lector_csv)
        for fila in lector_csv:
            nombre, stock = fila[0], int(fila[2])
            if stock < stock_minimo:
                productos_filtrados.append(nombre)
    return productos_filtrados

def generar_reporte(ruta_archivo_txt, lista_productos):
    """
    Toma una lista de productos y la escribe en un archivo de texto de
reporte.
    """
    with open(ruta_archivo_txt, 'w') as f_out:
        f_out.write("--- Productos con Stock Bajo ---\n")
        for producto in lista_productos:
            f_out.write(f"{producto}\n")
```

Paso 4: Importar y Usar en `main.py`

`main.py` (nuestro orquestador):

```
import procesamiento

# --- CONFIGURACIÓN ---
ARCHIVO_ENTRADA = 'inventario.csv'
ARCHIVO_SALIDA = 'productos_a_reponer.txt'
UMBRAL_STOCK = 20

# --- LÓGICA PRINCIPAL ---
try:
    productos =
procesamiento.filtrar_productos_por_stock(ARCHIVO_ENTRADA, UMBRAL_STOCK)
    procesamiento.generar_reporte(ARCHIVO_SALIDA, productos)
    print(f"Reporte '{ARCHIVO_SALIDA}' generado con éxito.")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo '{ARCHIVO_ENTRADA}'.")
```

```
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
```

Depuración: ¿Qué pasa si falla el `import`?

Un error muy común es el `ModuleNotFoundError`.

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    import proccesamiento
ModuleNotFoundError: No module named 'proccesamiento'
```

Causas y Soluciones:

1. **Error de Tipeo:** El 90% de las veces. Revisar si el nombre en `import` (`proccesamiento`) coincide *exactamente* con el nombre del archivo (`procesamiento.py`).
2. **Ubicación Incorrecta:** Python busca los módulos en el mismo directorio donde se ejecuta el script. Asegúrate de que `main.py` y `procesamiento.py` estén en la misma carpeta.

---

4. La Magia de `if __name__ == "__main__"`

Esta línea es una de las construcciones más importantes de Python. Permite que un archivo `.py` se comporte de dos maneras diferentes:

- Como un **script ejecutable**.
- Como un **módulo importable**.

Python asigna a una variable especial, `__name__`, el valor de `"__main__"` **solo** si ese archivo es el que se está ejecutando directamente. Si el archivo es importado, `__name__` valdrá el nombre del archivo (ej: `"procesamiento"`).

Demostración Práctica:

Agreguemos esto al final de nuestro `procesamiento.py`:

```
# ... (las funciones de antes) ...

# Este bloque solo se ejecutará si corremos "python procesamiento.py"
if __name__ == "__main__":
    print("¡Has ejecutado el módulo de procesamiento directamente!")
```

```
    print("Este es un buen lugar para poner pruebas unitarias o
demostraciones.")

    # Ejemplo de demostración:
    productos_demo = filtrar_productos_por_stock('inventario.csv', 10)
    print(f"Productos de demostración con stock < 10: {productos_demo}")
```

Ahora, veamos la diferencia en la terminal:

Ejecutamos `main.py` (que importa `procesamiento`):
> python main.py

Reporte 'productos_a_reponer.txt' generado con éxito.

1. *Resultado:* Solo se ejecuta la lógica de `main.py`. El bloque `if __name__ == "__main__"` de `procesamiento.py` se ignora, como debe ser.

Ejecutamos `procesamiento.py` directamente:
> python procesamiento.py

¡Has ejecutado el módulo de procesamiento directamente!

Este es un buen lugar para poner pruebas unitarias o demostraciones.

Productos de demostración con stock < 10: ['Producto X']

2. *Resultado:* Ahora sí se ejecuta el código dentro del `if`, permitiéndonos probar el módulo de forma aislada.

---

5. "Pilas Incluidas": La Biblioteca Estándar de Python

Python viene con una vasta colección de módulos listos para usar, sin instalar nada. Ya conocemos `csv` y `os`. Aquí hay una tabla de otros módulos fundamentales:

Módulo	Caso de Uso Rápido	Ejemplo
`datetime`	Para manejar fechas y horarios. Esencial para logs, reportes, etc.	`from datetime import datetime; ahora = datetime.now()`

`math`	Funciones matemáticas avanzadas (raíces, trigonometría, constantes).	`import math; math.sqrt(81)`
`random`	Generar números aleatorios, elegir elementos al azar, barajar listas.	`import random; random.randint(1, 6)`
`os`	Interactuar con el sistema operativo (manejar archivos, directorios).	`import os; os.path.join('reportes', 'ventas.txt')`
`json`	Para leer y escribir en formato JSON, muy común en APIs web.	`import json; json.dumps({'key': 'value'})`
`sys`	Acceder a parámetros del sistema, como los argumentos de la línea de comandos.	`import sys; archivo = sys.argv[1]`

---

6. El Vistazo al Futuro: Modularización en Proyectos Reales

La estructura que aprenden hoy no es un "ejercicio académico", es **exactamente la misma base que usan los proyectos profesionales a gran escala**.

- **Un videojuego simple:**
  - `main.py`: Bucle principal del juego.
  - `jugador.py`: Lógica del personaje principal.
  - `enemigos.py`: Lógica de los adversarios.
  - `motor_grafico.py`: Funciones para dibujar en pantalla.
- **Una aplicación web (backend):**
  - `app.py`: El servidor principal.
  - `rutas.py`: Define las URLs (`/home`, `/productos`).
  - `modelos.py`: Interactúa con la base de datos.
  - `autenticacion.py`: Maneja el login y los permisos.

Dominar la modularización es el paso clave para dejar de escribir pequeños scripts y empezar a pensar como un arquitecto de software.

---

7. Propuesta de Ejercicios Prácticos

Bloque 1: Creando y Usando Módulos

1. **Módulo de Geometría:**
    - Crea un archivo `geometria.py` con funciones `area_rectangulo(b, h)` y `perimetro_cuadrado(l)`.
    - Crea un `main.py` que importe `geometria` y lo use para calcular y mostrar el área de un rectángulo 10x5 y el perímetro de un cuadrado de lado 4.

Bloque 2: Aplicando la Modularización

2. **Refactorizando la Bitácora:**
    - Toma el ejercicio "Bitácora Personal" de la clase 8.
    - Divídelo en dos archivos:
        - `bitacora_utils.py`: Debe tener una función `agregar_entrada(texto)` que se encargue de la lógica de abrir `bitacora.txt` y añadir la entrada con fecha y hora (usa el módulo `datetime`).
        - `main.py`: Debe pedir el texto al usuario y llamar a la función `agregar_entrada`.

Bloque 3: Desafío Integrador (Combinando Módulos)

3. **Mini Sistema de Análisis de Texto:**
    - Crea un programa que analice un texto simple y dé estadísticas. Debe tener 3 archivos:
        - `analizador_texto.py`: Contendrá dos funciones. `contar_palabras(texto)` que devuelve el número de palabras, y `contar_caracter(texto, caracter)` que devuelve cuántas veces aparece un carácter.
        - `reportador.py`: Tendrá una función `generar_reporte(stats, fecha)` que toma un diccionario de estadísticas y una fecha, y las imprime de forma bonita en la consola.
        - `main.py`:
            1. Tendrá un texto fijo (ej: `"Este es un texto de prueba para el ejercicio."`).
            2. Importará los otros dos módulos.
            3. Usará `datetime` para obtener la fecha y hora actual.
            4. Llamará a las funciones de `analizador_texto` para obtener las estadísticas (número total de palabras y cuántas veces aparece la letra 'e').
            5. Guardará estas estadísticas en un diccionario.
            6. Finalmente, llamará a `generar_reporte` del módulo `reportador` para mostrar el resultado final.